BAYMAX bug review

Reviewed on **2026-09-28**, against the current working tree in `project-baymax`, including its existing uncommitted changes. Base commit: `43640c5`, branch `feat/phase1-scenario-loader`.

This report contains **23 findings**, ordered from most important to least: **5 high**, **16 medium**, and **2 low**. Priority reflects the project's emphasis on trustworthy evaluation, reproducible training, and correct local execution. No critical finding was established.

The review covered active code under `src/baymax`, related training/evaluation scripts and configurations, tests, and relevant project documents/ADRs. The separate `project-baymax-policy-pr` checkout, archived work, and legacy `task_manager` application were outside this review. Intentionally deferred features, such as real integrations, conversational memory, rollback execution, and deeper adapter-outcome scoring, are not counted as bugs.

Evidence labels: “Reproduced” means a local test or controlled probe demonstrated the behavior. “Code inspection” means the execution path establishes the defect, but the affected full workload was not run. No hosted inference, model training, or benchmark reruns were performed. Application code and historical artifacts were not changed.

Fix status (2026-09-28)

- BUG-002: fixed duplicate-content/contrast-group splitting; generator version 11. Added validation of content disjointness. Corrected local data is at `data/sft/v1/baymax-control-disjoint`; historical data was preserved.
- BUG-003: local parsing now rejects incomplete output, trailing output, invalid calls/arguments and conflicting meta-tool calls atomically. `parse_error` and `invalid_model_output` prevent successful scoring of a salvaged subset. Saved generations were rescored under `results/phase2/parser-integrity-v1/`.
- BUG-006: the active control config now pins the corrected disjoint dataset. Historical two-epoch/mix recipes have not been migrated automatically.
- BUG-012: the configured seed is now applied before model/LoRA construction. This does not retroactively make earlier training runs reproducible.

The remaining findings below are not marked fixed by this update. The original review evidence and historical results remain applicable to the earlier code.

High priority

BUG-001 — Sensitive values bypass error redaction

Locations: telemetry/logger.py, lines 23–35 and 111; core/dispatcher.py, line 91; core/agent.py, lines 246–253 and 274–279; service/mlx_backend.py, exception handling in both generation methods; service/api.py, lines 188–195.

The logger redacts dictionary keys, but exception and validation-error text is already a string by the time it reaches that filter. Pydantic error messages can contain the original input, including nested credentials. MLX additionally embeds raw exception text in `BackendError.message`, which the HTTP handler returns to the caller.

Reproduced: a fake `clipboard.write` call with `{"text": {"password": "FAKE_SECRET_SENTINEL"}}` wrote the sentinel into tool-error trace events. An MLX generation stub raising an exception containing another synthetic sentinel preserved it in the client-facing error message. No actual credentials were used. Separately, fields named `auth`, `key`, and `bearer` are not redacted despite ADR 0010 requiring those patterns.

Impact: malformed input or backend failures can disclose sensitive values in traces and error responses.

Suggested fix: sanitize structured validation errors before converting them to text, omit input values from diagnostics, use safe public backend-error messages, and implement the credential-key patterns specified by ADR 0010. Test the actual error paths as well as ordinary event dictionaries.

BUG-002 — Exact training examples leak into synthetic validation and test splits

Location: models/baymax_sft.py, lines 2366–2387.

Splitting groups examples by `contrast_group` or source ID. Different groups can contain identical complete SFT examples, so those copies can enter different partitions.

Reproduced: running the current generator and splitter in memory with the default seed produced:

Measurement	Observed
Generated rows	1,000
Unique complete SFT examples	750
Validation rows identical to a training row	75 / 200
Test rows identical to a training row	43 / 100

These are identical system, user, and assistant messages, not just similar tasks. For example, `baymax_generated_refuse_119` in training and `baymax_generated_refuse_059` in test have identical SFT content.

Impact: validation loss and any evaluation using the synthetic test partition are contaminated by training examples. This finding does **not** establish leakage into the separately authored final benchmark scenarios or quantify a change in benchmark scores.

Suggested fix: keep both duplicate-content groups and contrast groups together during splitting, and validate content disjointness across partitions. Version the corrected dataset and preserve existing experiment artifacts.

BUG-003 — The local evaluation parser deletes invalid calls before scoring

Location: eval/lora_runner.py, lines 235–238 and 270–273, plus `_extract_json_candidate`.

When `AgentToolCall` validation fails, the parser catches the error and continues. Consequently, malformed tool names disappear from the scored response. Extraction also accepts the first JSON value without accounting for additional emitted JSON calls.

Reproduced: a correct calendar-create call followed by `{"name": "delete_all_events", "arguments": {}}` was reduced to the correct call alone. A local scorer probe returned `task_success=True`, `hallucination_rate=0`, and no failure reasons. Naming that same unsupported extra call `calendar.delete_all` preserved it and correctly

produced failure with hallucination rate `0.5`.

Impact: the parser can make worse model output score better by erasing evidence of invalid or hallucinated calls. This affects an already implemented metric, rather than the documented deferred hallucinated-entity checks.

Suggested fix: preserve a parse/validation failure when any emitted call is invalid and account for trailing structured output. Do not weaken the tool-name contract or silently salvage a successful subset.

BUG-004 — Tool-result values never reach the final response model

Location: core/agent.py, lines 313–320.

`_render_action_log_summary()` passes the names of returned fields to `interpret_results`, but discards their values. The interpretation request contains the original user request and this summary, with no other route to the result data.

Reproduced: reading a clipboard containing a unique synthetic string yielded only:

```
1. clipboard.read() — OK → returned 1 field(s): text
```

The clipboard content was absent from the interpretation request.

Impact: the final model cannot accurately answer what was read or report generated IDs and returned details. Both inference backends receive the incomplete summary.

Suggested fix: include bounded, safely serialized result values in the interpretation input. Add a test that checks returned content reaches the backend, rather than merely checking that a tool call succeeded.

BUG-005 — Repeating a training run can overwrite an existing adapter

Location: models/sft_training.py, lines 180–181 and 398–400; entry point `run_sft_lora_training`.

The adapter destination is always `<output_dir>/adapter`. The entry point has no existing-artifact guard; the save path explicitly uses `mkdir(..., exist_ok=True)` followed by `save_pretrained()` into the same directory. Reusing a configuration or output override therefore reuses the previous adapter's filenames.

Evidence: code inspection. The destination calculation and write path were inspected. No existing adapter was overwritten and no training was run to demonstrate data loss.

Impact: a routine rerun can replace the checkpoint underlying an earlier experiment, contrary to the project's immutable model-version requirement. A recorded path can then identify different weights.

Suggested fix: reject occupied experiment/adapter destinations before training, or require a fresh versioned destination. Keep any explicit resume behavior distinct from starting a new run.

Medium priority

BUG-006 — The control training configuration pins the wrong dataset revision

Location: qwen2_5_1_5b_lora_baymax_control_v1.json, line 7; models/sft_training.py, lines 121–127.

The control configuration pins revision `342d766b20d61ab5056c404a2a8b3ce51a1b5cde6226fd182d1fc97fcd3ffe8c`. The current generator produces `8774e47a275fa035f240665c50967a0e4029056683e0971e3afd802ea38b5587`, also present in the current local split manifest.

Reproduced: configuration validation returns `valid=False` with a `data.revision` mismatch. The existing test `test_committed_qwen_lora_configs_are_valid[config_path1]` fails. Rebuilding with the current generator does not resolve this mismatch.

Impact: the supplied control run cannot pass training preflight against data generated by the current code.

Suggested fix: provide a versioned configuration matching the intended current dataset, or reproducibly retain the historical generator/data required by the older configuration. Keep revision validation intact. The untracked two-epoch mix configuration also mismatches its current local manifest; establish the intended mixing recipe before changing that record.

BUG-007 — The HTTP service hides supplied initial state from the planner

Location: service/api.py, lines 247–265; compare eval/agent_runner.py, lines 150–155.

`POST /invoke` initializes adapters from `req.initial_state` but passes only `req.context` to the agent. The evaluation runner includes initial state in planning context; the HTTP service does not.

Reproduced: invoking the service with a populated Notion task list and the default context produced `PlanRequest.context == {}`. The planner never received the supplied task's ID or title.

Impact: requests referring to entities in `initial_state` cannot be planned reliably through the service unless the caller manually duplicates that state in `context`. Service behavior diverges from the evaluated path.

Suggested fix: construct planning context consistently at both entry points, including the provided initial state with a documented precedence rule.

BUG-008 — Runtime failures are mislabeled as model refusal decisions

Location: core/contracts.py, lines 256–262; core/agent.py, lines 203 and 272.

`to_agent_response()` infers the decision from final `response_type`. Plan-validation and exhausted tool-execution failures set `TASK_REFUSAL`, even when the model selected an action plan.

Reproduced: a fake backend returned an ACT plan containing an unavailable tool. Runtime validation rejected it and exported `decision_type="refuse"`; the decision scorer recorded an incorrect refusal rather than the model's selected action.

Impact: decision accuracy and failure analysis mix model policy decisions with runtime errors.

Suggested fix: retain the selected model decision independently from execution outcome and response wording. Exercise both validation and dispatch failures in decision-metric tests.

BUG-009 — One backend failure discards completed evaluation work

Location: eval/agent_runner.py, lines 191–200; eval/cli.py, lines 296–312.

A `BackendError` escapes the scenario loop. The CLI writes its artifact only after that loop returns, so completed results are never saved if a later scenario fails.

Reproduced: a three-scenario fake backend sequence containing a successfully returned clarification, a `BackendError`, and another response made two calls, left the third scenario unrun, and made zero artifact writes.

Impact: transient or exhausted backend failures lose already completed evaluation results and cost records, and provide no structured failure record for the failed scenario.

Suggested fix: preserve partial results and represent infrastructure failures explicitly, consistent with ADR 0011. Do not fabricate an agent response or count a backend outage as a model refusal.

BUG-010 — Local evaluation artifacts omit checkpoint and generation identity

Location: eval/cli.py, lines 360–365 and 422–425.

The local runner uses `adapter_path`, `max_new_tokens`, and `temperature`, but none is passed into the result payload. The backend name identifies only `local-lora:<base_model>`, while `benchmark_config` contains the runner and fake-adapter flag.

Evidence: code inspection and existing artifact headers. Two adapters or generation configurations can produce artifacts with identical configuration metadata. Filename conventions are the remaining clue, rather than a self-contained run record.

Impact: result contents cannot reliably identify or reproduce the model experiment that generated them.

Suggested fix: persist checkpoint identity and effective generation settings, together with relevant dataset/scenario and configuration identity. Add fields to new artifacts without rewriting historical results.

BUG-011 — Missing scenario directories silently shrink an evaluation suite

Location: eval/scenario_loader.py, line 166 and lines 187–197.

The loader calls `directory.glob("*.json")` without checking that each requested path exists and is a directory. A nonexistent directory behaves like an empty suite.

Reproduced: loading `[Path("scenarios/v1"), Path("__review_nonexistent_scenario_directory__")]` returned 50 scenarios normally.

Impact: a typo in a second suite path silently changes benchmark composition while still allowing a normal result artifact.

Suggested fix: validate every supplied path before loading. Preserve the existing, explicitly tested behavior for an empty directory that actually exists.

BUG-012 — The training seed is set after random adapter initialization

Location: models/sft_training.py, lines 356, 386, and 418.

`peft.get_peft_model()` initializes LoRA parameters before the configured seed reaches `Trainer`. There is no earlier RNG initialization. In installed Transformers 5.15.0, the trainer seeds during its constructor, after the adapter has already been created.

Reproduced: constructing adapters on identical copies of a tiny Qwen model under two ambient RNG states, then applying the same configured seed, produced different `q_proj.lora_A` and `v_proj.lora_A` weights. This was an offline initialization probe, without optimization or downloaded weights.

Impact: identical recorded seeds and configurations do not reproduce initial adapter weights.

Suggested fix: seed before model/adapter construction, or use a model initialization callback whose seeding is controlled by the trainer.

BUG-013 — Training configuration paths are Windows-specific

Locations: qwen2_5_1_5b_lora_v1.json, lines 11–18 and runtime paths; equivalent fields in the control, mix, and two-epoch mix configurations; models/training_config.py, lines 35–40 and 138–141.

The four current configurations serialize paths with backslashes, such as `data\sft\v1\xlam\train.jsonl`. Deserialization uses ordinary `Path` handling without translating Windows separators.

Reproduced at the path-semantics level: `PurePosixPath(value).parts` treats that value as one literal filename. No Linux or macOS training run was performed.

Impact: on Linux/macOS, preflight searches for filenames containing backslashes instead of datasets generated under normal directories. Output destinations have the same portability defect.

Suggested fix: serialize portable forward-slash paths and make path/configuration hashing consistent across platforms.

BUG-014 — Three configured backend attempts can make nine HTTP requests

Location: service/inference.py, line 469 and `_call_llm`.

BAYMAX wraps API calls in its three-attempt retry loop, but constructs `AsyncOpenAI` without disabling the SDK's own retries. The installed SDK defaults to two retries per call.

Reproduced: an in-memory HTTP transport returning status 500 received **nine requests**, while BAYMAX reported **three attempts**. Sleeps were mocked and no network request was made.

Impact: rate limits and outages cause more traffic and latency than the documented retry budget; attempt telemetry understates actual HTTP attempts.

Suggested fix: make one layer responsible for retries, for example by passing `max_retries=0` to the SDK client when BAYMAX's wrapper is active.

BUG-015 — The MLX planner never receives tool argument schemas

Location: service/mlx_backend.py, lines 232–248.

`_build_plan_prompt()` includes tool names but ignores `PlanRequest.tool_schemas`. The OpenAI backend and local Hugging Face evaluation path both expose parameter specifications.

Reproduced: a request containing a uniquely named required schema property produced an MLX prompt without that property or any tool parameter schema.

Impact: the local service model must guess required argument names and types, and receives materially different tool information from the other evaluated paths.

Suggested fix: include the supplied tool specifications in the MLX planning prompt, with formatting based on the training interface rather than individual benchmark fixtures. Validate the prompt with a schema-containing request.

BUG-016 — Service backends mishandle malformed model arguments

Locations: service/inference.py, lines 536–579; service/mlx_backend.py, lines 295–307.

Parsed model output is used before its shape is validated. OpenAI meta-tool handling calls `.get()` on decoded values even when they are lists or scalars. MLX does the same for call entries and arguments. OpenAI additionally replaces invalid JSON with `{}` instead of reporting invalid output.

Reproduced: OpenAI `request_clarification` with arguments `[]` raised `AttributeError`. MLX refusal arguments containing a nonempty list or integer did likewise. OpenAI `clipboard__read` with arguments `{not-json` became a valid-looking `clipboard.read({})` plan.

Impact: malformed output causes unstructured request failures or is silently changed into executable output. These failures occur outside the intended generation-error handling.

Suggested fix: validate call objects and argument dictionaries before access, and return a structured parse/backend failure for invalid output. Do not turn invalid arguments into empty arguments.

BUG-017 — MLX JSON extraction breaks on valid strings containing `}]`

Location: service/mlx_backend.py, lines 269–289.

The non-greedy regular expression used to find a JSON array does not understand JSON strings or nesting. It stops at matching characters inside string content.

Reproduced: this valid model output became a clarification request instead of a clipboard write:

```
[{"name": "clipboard.write", "arguments": {"text": "literal }] marker"}}]
```

Impact: ordinary text/code that happens to contain those characters can make a valid tool call fail parsing.

Suggested fix: use JSON-aware decoding that respects strings and nesting. Cover both embedded delimiter text and nested JSON in parser tests.

BUG-018 — Evaluation silently ignores invalid clipboard-read arguments

Location: eval/fake_executor.py, lines 81–82; compare core/dispatcher.py, line 85.

The evaluation dispatcher invokes `self._clipboard.read()` without forwarding arguments. The service dispatcher forwards them and consequently rejects unexpected keyword arguments.

Reproduced: `clipboard.read` with `{"unexpected": "x"}` returned `success=True` in evaluation and `success=False` with a `TypeError` through the service dispatcher.

Impact: the two execution paths disagree for the same malformed model call, despite the shipped tool schema disallowing additional properties.

Suggested fix: forward and validate arguments consistently, preferably through shared dispatch behavior. This is separate from intentionally deferred scoring of tool-execution outcomes.

BUG-019 — The documented HTTP server command lacks its dependency

Locations: pyproject.toml, dependencies; service/api.py, line 11; `uv.lock`.

The service documents `uv run uvicorn baymax.service.api:app --reload --port 8000`, but Uvicorn is neither declared nor present in the lockfile. FastAPI is installed without an extra supplying the server.

Evidence: dependency/lockfile inspection and a local import-availability probe, which found no `uvicorn` module in the project environment. No server startup was attempted.

Impact: installing the documented locked dependencies is insufficient to run the documented service command.

Suggested fix: explicitly provide the server dependency or document and supply a supported installation extra containing it.

BUG-020 — `create_app()` creates an application without the BAYMAX endpoints

Location: service/api.py, lines 166–175, 218, and 237.

The factory registers exception handlers, but `/health` and `/invoke` are registered afterward only on the module-level `app` instance.

Reproduced: `TestClient(create_app()).get("/health")` returned 404. The factory application's routes contained only OpenAPI/documentation endpoints. Existing endpoint tests construct another app and manually re-register routes, which hides this defect.

Impact: callers using the advertised application factory, including a factory-based server launch, get no BAYMAX routes. The documented module-level `:app` launch is not affected by this particular bug.

Suggested fix: register the router/endpoints inside the factory and have the module-level app use that same construction path. Test the actual factory.

BUG-021 — MLX generation blocks the asynchronous service loop

Location: service/mlx_backend.py, lines 160–170 and 200–210.

Both async backend methods directly call synchronous `self._generate(...)`. No executor or other offloading is used, despite ADR 0011 specifying wrapping synchronous inference.

Reproduced with a synchronous generation stub: an independently scheduled async heartbeat ran only after generation finished. No Apple Silicon hardware or MLX model was used for this scheduling probe.

Impact: during local generation, the service event loop cannot process health requests or other waiting asynchronous work. Delay follows generation duration.

Suggested fix: run synchronous inference through a suitably serialized worker/executor, respecting the model runtime's thread-safety requirements.

Low priority

BUG-022 — Result task IDs do not match their runtime traces

Locations: eval/agent_runner.py, line 173; core/agent.py, line 109.

The result synthesizes `<run_id>.<scenario_id>.0`, while the agent records `<run_id>.1.0`. All scenarios given the same run ID therefore also reuse one runtime task ID.

Reproduced: a captured trace contained `review.1.0`, while its corresponding scenario result contained `review.calendar_create_001.0`.

Impact: task-ID-based joins between results and traces fail or become ambiguous. The separate trace ID reduces the severity but does not make the task identifier correct.

Suggested fix: assign each scenario a consistent unique task identity and export the identity actually recorded in telemetry.

BUG-023 — Warmup is calculated before rows without training labels are removed

Location: models/sft_training.py, lines 455–465 and 475–480.

Tokenization removes examples with no trainable assistant labels, but the `warmup_steps` fallback uses the original loaded row count. The fallback is active with installed Transformers 5.15.0, whose `TrainingArguments` no longer accepts `warmup_ratio`.

Reproduced: tokenizing the first 1,000 local xLAM training rows with the cached tokenizer, low-VRAM settings, and a 512-token limit retained 267 rows. Those settings yield 17 optimizer steps, but the fallback computed two warmup steps instead of `ceil(17 × 0.03) = 1`. No training was performed.

Impact: truncation/filtering changes the effective warmup schedule away from the configured ratio.

Suggested fix: calculate warmup from the retained training dataset and effective step count, or use the installed trainer's supported ratio mechanism.

Verification performed

Checks used the existing virtual environment without installing or upgrading dependencies:

Check	Result
`.venv/Scripts/python.exe -m ruff format --check`	Passed; 108 files already formatted
`.venv/Scripts/python.exe -m ruff check .`	Passed
`.venv/Scripts/python.exe -m pyright --pythonpath .venv/Scripts/python.exe`	Passed; 0 errors, 0 warnings
`.venv/Scripts/python.exe -m pytest -q -p no:cacheprovider --basetemp <fresh-workspace-temp-directory>`	**289 passed, 1 failed, 1 skipped**

The failing test is the control-config revision mismatch in BUG-006. The skipped test requires explicitly enabled live OpenAI integration. Initial attempts using `uv` and default temporary paths encountered environment restrictions; the checks above used direct virtual-environment executables, an explicit interpreter for Pyright, and a fresh writable temporary directory for pytest. Those environment errors are not listed as project bugs.

Additional verification used in-memory fake backends, synthetic error values, a mocked HTTP transport, the actual synthetic dataset generator/splitter, installed dependency source, and a cached tokenizer. No paid requests or full training runs were made. Full MLX execution on Apple Silicon and POSIX training were not available in this Windows review.

The passing checks do not establish that the behavioral findings are covered: several reproductions exercise paths the existing tests bypass or do not assert. This report records the defects established during the review; it is not a guarantee that no other bugs remain.

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.